



Ingredio

Find recipes from what you already have

Mobile Applications · THWS / TAMK · 2026

Sofia Khyzhnychenko

Anastasia Pylova

Halil Hakan Karabay

Armanc Beler

What is Ingredio?

The problem

You open the fridge, see some ingredients, and have no idea what to cook. You either waste food or order takeout.

Our solution

Select the ingredients you have. Ingredio instantly fetches and ranks real recipes by how many of your ingredients they need — no guesswork, no waste.

7

Screens

9

Riverpod providers

10+

Tests

10

Packages used

Key Features



Smart Pantry

Search, select & quantity-tag up to 30 ingredients with debounced live search



Recipe Discovery

Recipes ranked by match count — the more ingredients you have, the higher it ranks



Favorites

Save any recipe locally via Hive for offline access



Profile & Onboarding

Name registration, editable profile, recipe collections, dietary prefs



Offline Support

Full Hive caching — ingredient list and recipes work without internet



Share Recipes

Native share sheet to send recipe details to any app

Architecture

Clean Architecture — 3 decoupled layers



Presentation

screens/ · widgets/ · providers/

Flutter widgets + Riverpod AsyncValue



Domain

entities/ · usecases/ · interfaces/

Pure Dart — zero Flutter dependencies



Data

api/ · local/ · repositories/

MealDB API (Dio) + Hive local cache

Dependency Injection via `get_it` — domain interfaces resolved to data implementations at startup

State Management

flutter_riverpod 2 — AsyncValue for every async operation

Provider	Type	Purpose
<code>recipesProvider</code>	<code>FutureProvider</code>	Matched recipes list
<code>recipeDetailProvider</code>	<code>FutureProvider.family</code>	Single recipe by ID
<code>ingredientsListProvider</code>	<code>FutureProvider</code>	Full ingredient list from API/cache
<code>ingredientsProvider</code>	<code>StateNotifierProvider</code>	Selected ingredients set
<code>ingredientQuantitiesProvider</code>	<code>StateNotifierProvider</code>	Quantity per ingredient
<code>favoritesProvider</code>	<code>StateNotifierProvider</code>	Persisted favourites
<code>connectivityProvider</code>	<code>FutureProvider</code>	Network state
<code>userProfileProvider</code>	<code>StateNotifierProvider</code>	Registered user name
<code>hiveDatabaseProvider</code>	<code>Provider</code>	Hive singleton

App Screens

Splash

Reads Hive on startup; routes to Register or Main

Register

Name input with validation; persisted to Hive; dark/light theme

Pantry

Ingredient list, live search, selection, quantities

Discover

Recipes ranked by match count + client-side search filter

Recipe Detail

Instructions, measures, YouTube link, share button

Favorites

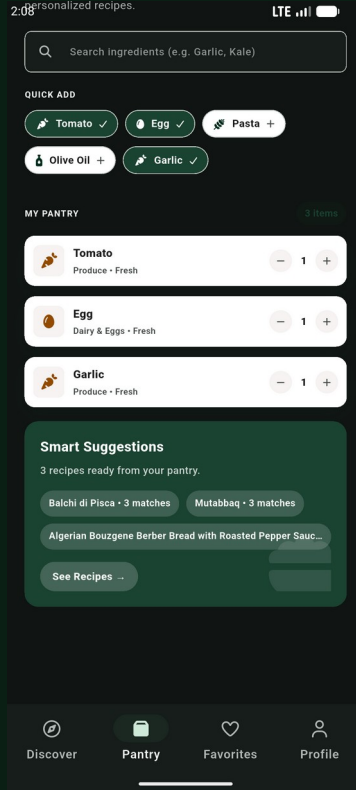
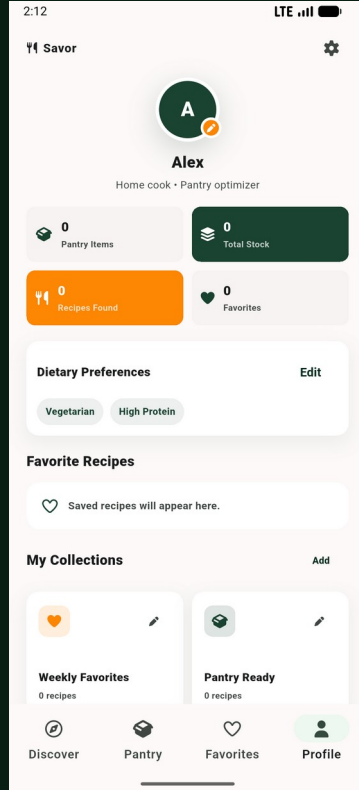
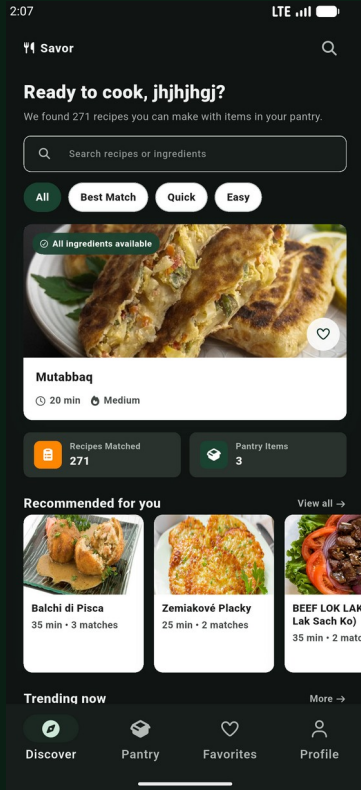
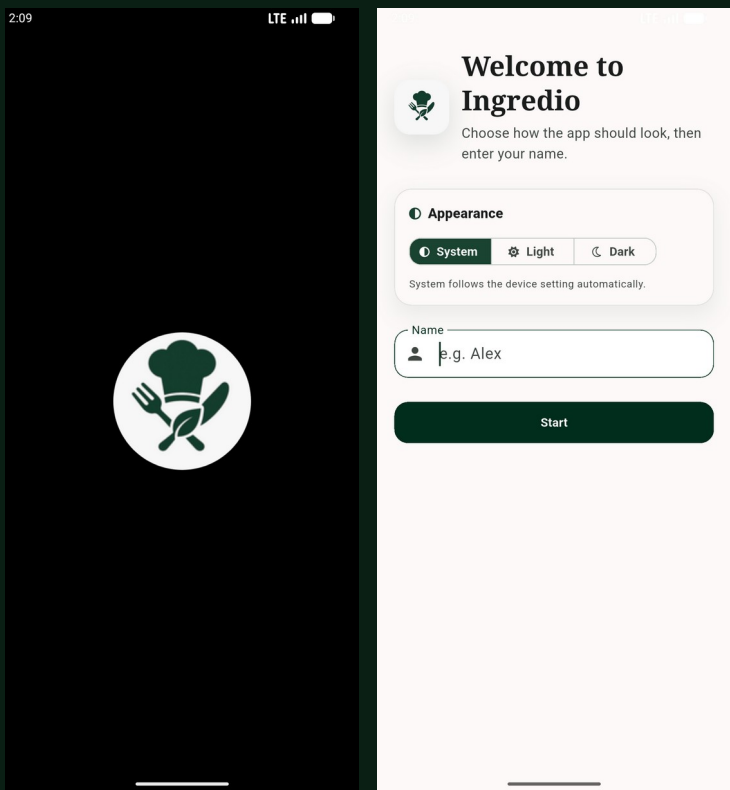
Offline recipe list from Hive

Profile/Settings

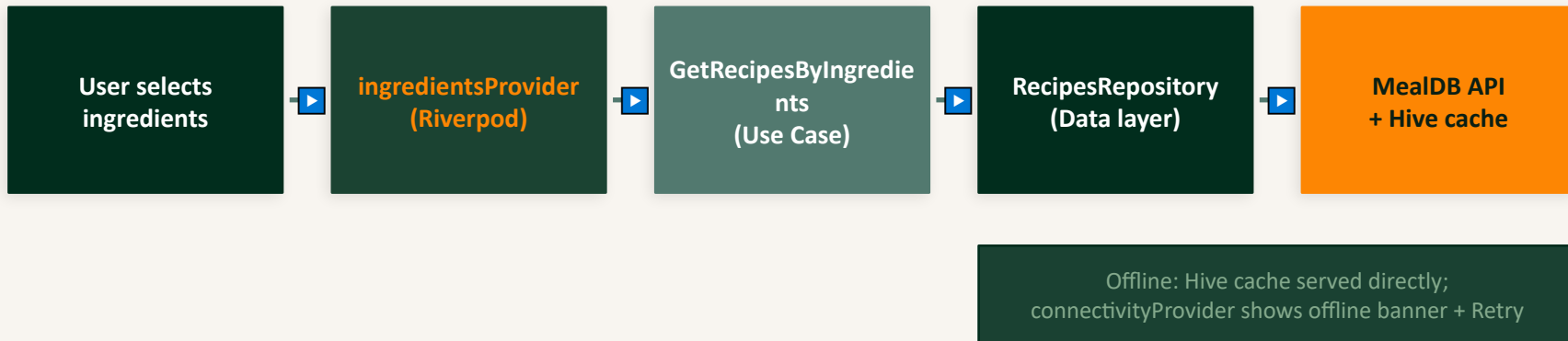
User name, collections, prefs, logout / delete all dark/light theme

Navigation: `Navigator.push(MaterialPageRoute)` · Bottom tab: `IndexedStack` with lazy loading · iOS: Custom CupertinoButton bar

App Screens



Data Flow



AsyncValue pattern — every screen handles all 3 states:

loading:

CircularProgressIndicator shown centrally

error:

Error message + Retry button, calls `ref.invalidate()`

data:

Content rendered; empty state handled gracefully

Testing

flutter test — all passing

Unit Tests



ingredient_icons_test.dart

IngredientIcons.forName() — icon & category mapping for poultry, spices, unknowns



get_recipes_by_ingredients_test.dart

Empty input returns early · Deduplication of ingredients · onlyBasicInfo flag passthrough



get_recipe_details_test.dart

GetRecipeDetails delegates to repository correctly

Widget Tests



recipes_list_screen_test.dart

Renders featured + recommended sections; search bar filters results



ingredients_screen_test.dart

Renders ingredient list; selection toggle



recipe_detail_screen_test.dart

Renders recipe name, instructions, action buttons



profile_screen_test.dart

Renders user name and profile sections

Tech Stack & Packages

Package	Why we use it
<code>flutter_riverpod ^2.0.0</code>	State management — providers, notifiers, AsyncValue
<code>get_it ^7.2.0</code>	Service locator / dependency injection
<code>dio ^5.0.0</code>	HTTP client — MealDB API calls
<code>hive + hive_flutter</code>	Lightweight local NoSQL — offline caching
<code>cached_network_image</code>	Efficient URL image loading + disk cache
<code>connectivity_plus</code>	Detect network state
<code>share_plus</code>	Native share sheet
<code>url_launcher</code>	Open YouTube links in system browser
<code>font_awesome_flutter</code>	Icon set used throughout the UI



Thank you!

Questions welcome

Sofiia Khyzhnychenko · Anastasia Pylova · Halil Hakan Karabay · Armanc Beler

github.com/soffffises/Ingredio

Mobile Applications · THWS / TAMK · 2026